CronJovi

Design document: Shortcuts-style automation for the Linux desktop

Author: Justin Garza · Status: Current for v0.0.1 · October 31, 2026

1. Summary

CronJovi is a local, visual automation app for the Linux desktop, modeled on Apple Shortcuts. A workflow is a trigger plus an ordered list of actions. It is not AI: it is plain, predictable automation that runs on your machine.

The app targets Wayland compositors first (niri, Hyprland) and keeps every workflow as a plain YAML file, so the GUI is optional and workflows can live in git.

2. Problem and gap

iPhone and Mac make small automations easy: pick a trigger, snap actions together, done. Linux has the pieces but no equivalent product. Existing tools fall into three groups:

Tool	What it does well	Why it falls short
n8n, Node-RED	Visual flow builders, big node libraries	Built for web/API work; desktop tasks need exec nodes
Home Assistant	Shortcuts-like editor	Devices and smart home only
Actiona, AutoKey	Desktop action builders	X11-oriented; limited on Wayland
Espanso, keyd, sxhkd	Snippets, remapping, hotkeys	No logic, no GUI, single purpose

Why the gap exists: Shortcuts works because Apple controls the OS and apps expose actions to it. Linux has no common action API, and Wayland blocks global hotkeys, input injection and window control by design. CronJovi works around this with small, well-known tools behind a consistent action model.

3. Goals and non-goals

Goals

- Shortcuts-level ease: snap blocks together, run, see what happened.
- Desktop-level actions: clipboard, notifications, windows, media, keystrokes, files.
- Wayland first. Triggers that work without X11 global hotkeys.
- Trustworthy scheduling: clear behavior for missed runs and overlap.
- Plain-file workflows (YAML) that can be versioned and shared.

Non-goals (v1)

- No AI or natural-language workflow generation.
- No cloud sync or accounts.
- No mobile app or remote control.

4. Architecture

4.1 Components

- **Engine (Python).** Loads YAML, runs actions in order, passes variables between steps, handles if/else, repeat and errors.
- **Action plugins.** Each action is a small function with typed inputs and outputs. Adding an action means adding one file.
- **Trigger layer.** Turns hotkeys, timers, file changes and system events into engine runs.
- **CLI.** `cronjovi run | list | new | logs`. The CLI ships first and is the contract the GUI builds on.
- **Service.** The engine runs as a systemd user service.
- **GUI.** A local web UI (Flask and plain JavaScript) in a WebKitGTK window, with a block editor. It reads and writes the same YAML.

4.2 Triggers

Trigger	Mechanism	Notes
Hotkey	Compositor binding runs `cronjovi run <name>`	Wayland has no portable global hotkeys
Schedule	systemd user timers	Missed-run behavior is explicit (see 5)
File change	inotify (watchdog)	Folder or glob
Device / power	udev and D-Bus	USB plugged in, battery below N%
Webhook	Small local Flask endpoint	Optional, off by default

4.3 First actions (21 ship in v0.0.1)

Action	Backed by
Run shell command	bash or user-selected shell, with timeout
Show notification	notify-send
Open app or URL	xdg-open
Get or set clipboard	wl-copy, wl-paste
Type text or keys	wtype or ydotool
HTTP request	Python requests
Move, copy, rename file	Python pathlib/shutil

Action	Backed by
Media control	playerctl
Wait	Engine timer
Set variable	Engine

4.4 Workflow file format

One YAML file per workflow. Variables from one step are available to later steps.

```
name: Focus Mode
trigger:
  hotkey: Super+Shift+F
steps:
  - action: dnd
    state: on
  - action: media
    command: pause
    players: all
  - action: shell
    run: niri msg action focus-workspace 3
    timeout: 10s
    stop_on_error: true
  - if:
      time_of_day: { after: "18:00" }
    then:
      - action: brightness
        percent: 40
    else:
      - action: brightness
        percent: 70
  - action: notify
    text: Focus mode on
```

5. Scheduling semantics

Scheduled workflows are where automation tools lose trust. CronJovi spends its simplicity budget on making these cases visible choices rather than hidden defaults.

5.1 When the machine was off or asleep

Every scheduled workflow shows an explicit choice:

- **Skip it.** Wait for the next scheduled time. History records “skipped: machine was off”.
- **Run once when the machine is back (default).** Several missed ticks still mean one run. CronJovi never replays every missed tick, because that would be a nasty surprise for anything that sends messages.

5.2 When the previous run is still going

- **Skip the new run (default).** History records “skipped: previous run still active” instead of another vague green status.
- **Stop the old run, start fresh.** Sends SIGTERM to the old run first.

5.3 History

Each run records four things: scheduled time, actual start, finish, and the command output. A skipped run still gets a row with the reason. Failures show the exit code and the captured output so the cause is visible without opening a terminal.

6. Interface

Desktop window, dark libadwaita-style look, sidebar plus content plus action library. The accent color is a single tweakable value.

6.1 Workflow editor

Sidebar groups workflows by trigger type. The center shows blocks in the order they run, like Shortcuts: a trigger, then actions, with an If/Otherwise branch indented. Blocks expand in place for details (the shell block is expanded below). The run log sits at the bottom and the action library on the right. A Blocks/YAML toggle shows the same workflow as a file. The status bar reports which Linux tools are installed and flags missing ones (here, brightnessctl).

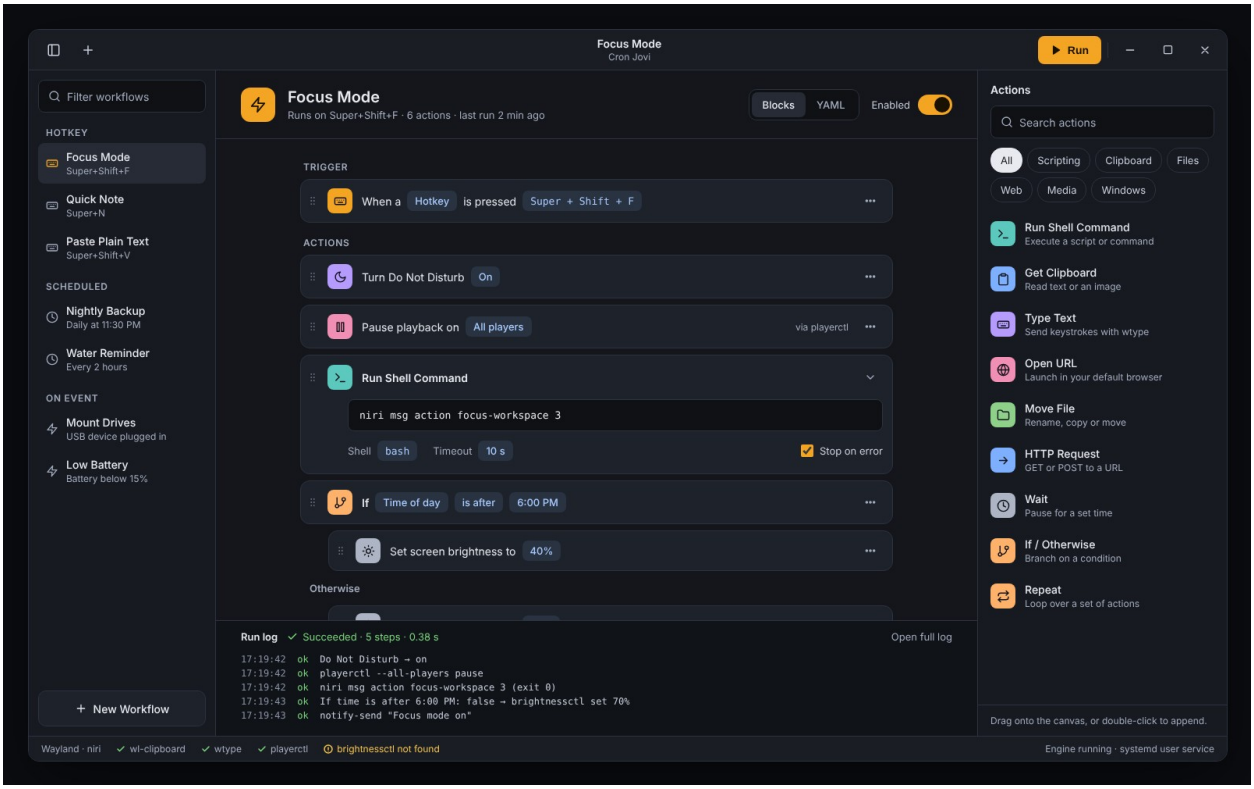


Figure 1. Workflow editor: Focus Mode (the steps pane scrolls; the last step is below the fold).

6.2 Library

A grid of workflow cards with folders, favorites and filters by trigger type. Each card shows its trigger and action count, with a play button to run it.

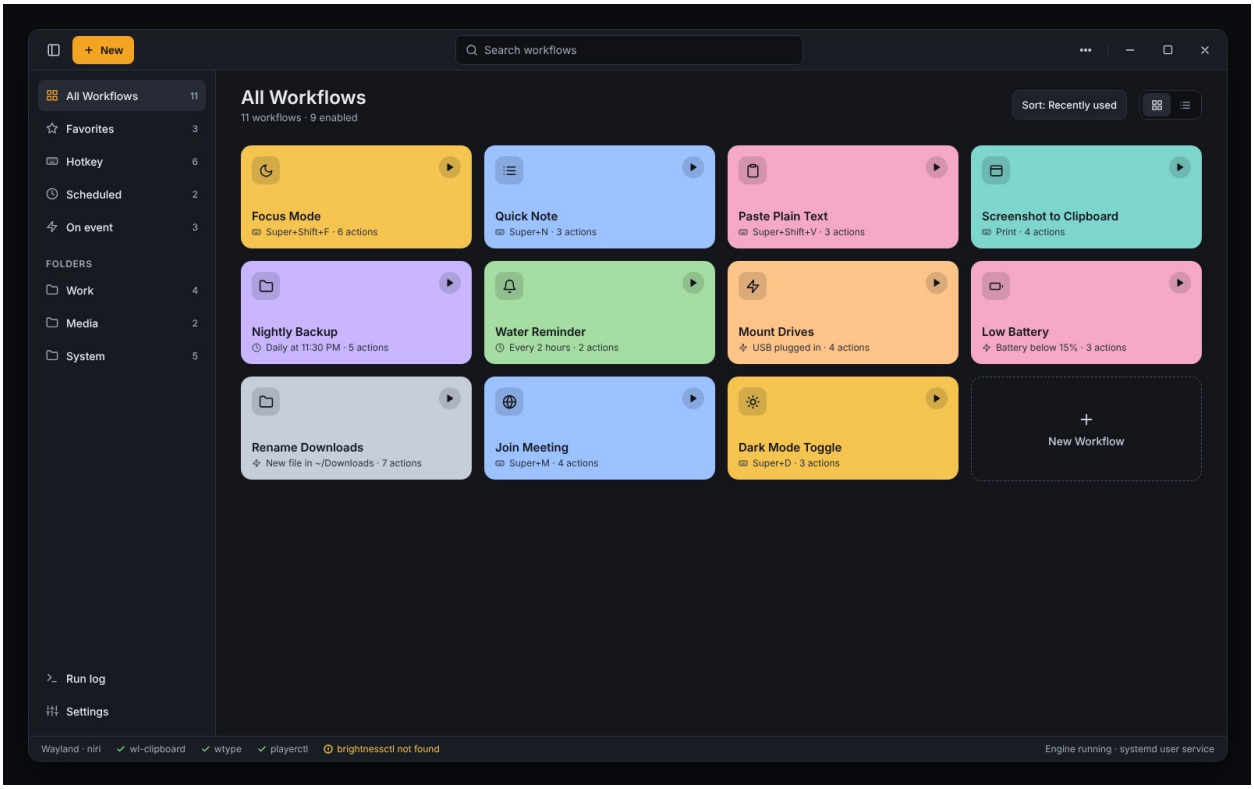


Figure 2. Library view with 11 workflows.

6.3 Scheduled workflow with history

The Nightly Backup demo covers the trust cases: the missed-run choice, the overlap choice, and a history table with scheduled time, actual start, duration, result and a note. The selected failed run shows the command output for an unwritable destination.

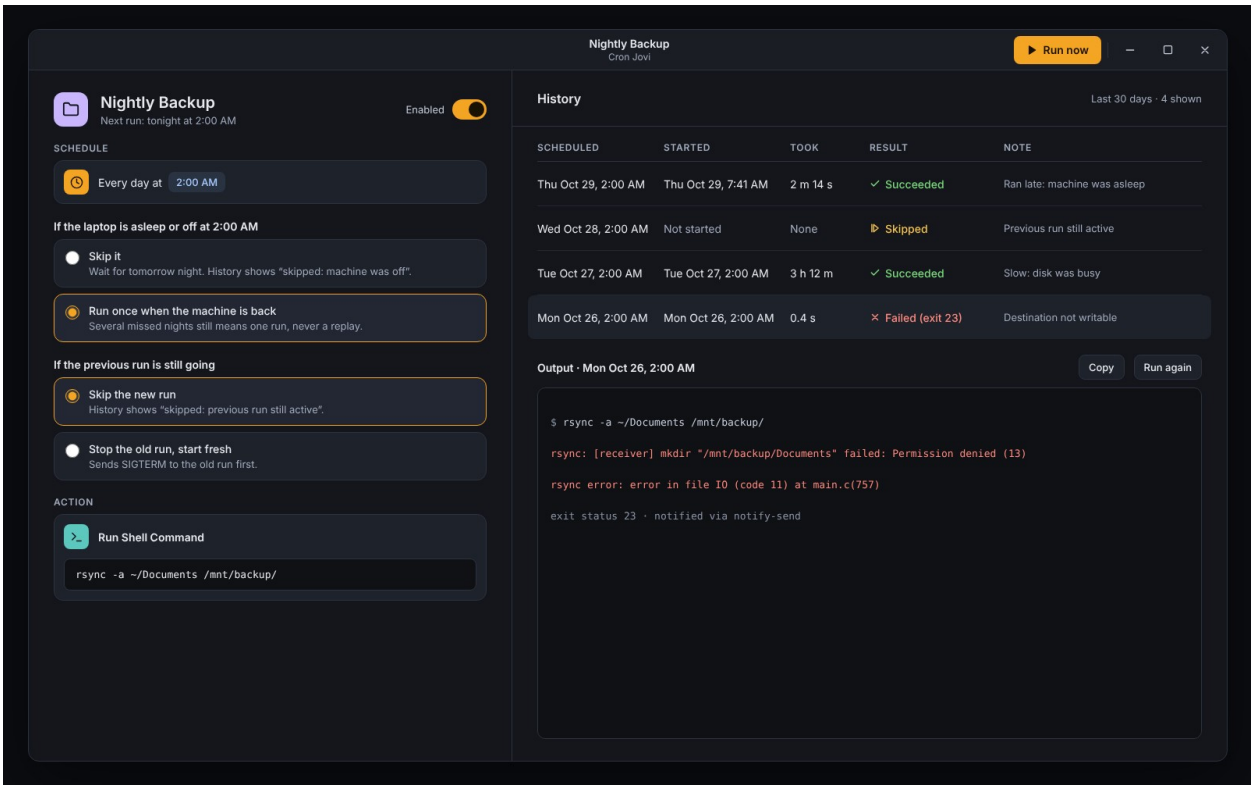


Figure 3. Scheduled workflow: missed-run and overlap settings with run history.

7. MVP and roadmap

MVP

- CLI plus YAML workflows.
- 21 actions (section 4.3).
- Hotkey and timer triggers.
- Run history with scheduled time, start, finish and output.

First demo

One folder-backup workflow that shows two real failure modes: the laptop asleep at the scheduled time, and an unwritable destination. This teaches more about the concept than adding another integration.

Later (all shipped in v0.0.1)

- GUI block editor (local web UI).
- File-change, device and power triggers.
- Webhook trigger.
- Folder or git repo as a workflow "gallery" for sharing.

8. Naming and repo

Item	Value
App name	CronJovi (shown as “Cron Jovi” in the UI)
Repo and folder	cron-jovi
Binary / CLI	cronjovi
Python package	cron_jovi
Alternates considered	Rube, Macro Polo, Doohickey, Thingamabot, Flowbert

```
cron-jovi/
├── cron_jovi/
├── workflows/
├── pyproject.toml
└── README.md
```

9. Risks and open questions

- **Wayland limits.** Input injection and window control depend on compositor support (wtype/ydotool, niri/Hyprland IPC). Plan per-compositor adapters.
- **Overlap with existing tools.** Checked: n8n, Node-RED and Huginn do web work, and Home Assistant does devices. None targets the Wayland desktop.
- **Safety of shell actions.** Workflows run arbitrary commands. Shared workflows need a review step before first run.
- **Sleep and wake detection.** Missed-run handling needs a reliable signal from systemd/logind.
- **GUI toolkit.** Decided: a local web UI for speed of building, shown in a WebKitGTK window.